

Федеральное государственное автономное образовательное учреждение
высшего образования

**«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО»**

Отчет

по лабораторной работе №2.3

«Алгоритмы на графах: BFS, DFS и Косарайю»

по дисциплине «Алгоритмы и структуры данных»

Автор: Бен Шамех Абделаиз

Факультет: ПИН

Группа: K3239

Преподаватель: Ромакина Оксана Михайловна

Санкт-Петербург 2026

1. Цель работы

Цель лабораторной работы — изучение базовых алгоритмов поиска на графах: поиска в ширину (BFS) для проверки достижимости, алгоритма Косарайю для нахождения компонент сильной связности, алгоритма поиска минимальной К-связности через 0-1 BFS.

2. Задание на лабораторную работу (Вариант 1)

- Задача 1. Лабиринт — проверить существование пути между u и v (вывод 1 или 0). [1 балл]
- Задача 5. Город с односторонним движением — число компонент сильной связности. [1.5 балла]
- Задача 17. Слабая К-связность — минимальное K для слабой К-связности графа. [4 балла]
- Задача 13*. Грядки — подсчёт числа грядок на прямоугольной сетке (BFS, доп.). [3 балла]
- Задача 15*. Герои — сумма подвесок мушкетёров, достигших королевы за L минут (BFS, доп.). [3 балла]

3. Краткие теоретические сведения

Поиск в ширину (BFS) — обход графа по уровням. Использует очередь FIFO. Для невзвешенных графов BFS даёт кратчайшие пути по числу рёбер. Сложность $O(V+E)$.

Алгоритм Косарайю находит компоненты сильной связности (SCC) за два прохода DFS: (1) DFS по исходному графу, запись порядка завершения; (2) DFS по транспонированному графу в обратном порядке. Сложность $O(V+E)$.

Слабая К-связность: 0-1 BFS (двусторонняя очередь). Рёбра по направлению — стоимость 0, против направления — стоимость 1. Минимальная $K = \max$ расстояний от всех вершин до всех.

4. Выполнение работы

4.1. Задача 1 — Лабиринт (BFS, поиск пути)

BFS от стартовой вершины. Если конечная вершина достигнута — выводим 1, иначе 0. Граф неориентированный, представлен списком смежности. Сложность $O(V+E)$.

Листинг 1 — task1.py

```
def find_path(n, edges, start, end):
    graph = [[] for _ in range(n+1)]
    for u, v in edges:
        graph[u].append(v)
        graph[v].append(u)
    visited = [False]*(n+1)
    queue = deque([start])
    visited[start] = True
    while queue:
        current = queue.popleft()
        if current == end:
            return True
        for neighbor in graph[current]:
            if not visited[neighbor]:
                visited[neighbor] = True
                queue.append(neighbor)
    return False
```

●●● Рисунок 1 — Задача 1: Лабиринт

\$ python3 task1.py	
Тест 1: n=4, start=1, end=4	Тест 2: n=4, start=1, end=4
Рёбра: 1-2, 3-2, 4-3, 1-4	Рёбра: 1-2, 3-2
BFS от 1: 2,4,3 — 4 достигнута	BFS от 1: только 2
output.txt: 1	output.txt: 0

Рисунок 1 — Демонстрация работы: Задача 1 (Лабиринт)

4.2. Задача 5 — Город с односторонним движением (SCC, Косарайю)

Алгоритм Косарайю: итеративный DFS по исходному графу (порядок завершения), затем итеративный DFS по транспонированному графу в обратном порядке. Каждый запуск второго DFS — новая SCC. Выводим число SCC.

Листинг 2 — task5.py

```

def kosaraju(n, edges):
    graph = defaultdict(list)
    rev_graph = defaultdict(list)
    for u, v in edges:
        graph[u].append(v)
        rev_graph[v].append(u)
    visited = [False]*(n+1); order = []
    def dfs(u):
        stack = [u]
        while stack:
            node = stack.pop()
            if not visited[node]:
                visited[node] = True
                for nb in graph[node]:
                    if not visited[nb]: stack.append(nb)
                order.append(node)
    for u in range(1, n+1):
        if not visited[u]: dfs(u)
    visited = [False]*(n+1); components = 0
    for u in reversed(order):
        if not visited[u]:
            stack=[u]; visited[u]=True
            while stack:
                node=stack.pop()
                for nb in rev_graph[node]:
                    if not visited[nb]:
                        visited[nb]=True; stack.append(nb)
            components += 1
    return components

```

●●● Рисунок 2 — Задача 5: Компоненты сильной связности

```

$ python3 task5.py
Тест 1: n=4, рёбра: 1->2, 4->1, 2->3, 3->1
Цикл 1->2->3->1: SCC={1,2,3}, отдельно {4}
output.txt: 2
Тест 2: n=5, без циклов -> каждая вершина = своя SCC
output.txt: 5

```

Рисунок 2 — Демонстрация работы: Задача 5 (SCC, Косарайю)

4.3. Задача 17 — Слабая К-связность

Для каждой пары вершин (s,v) находим минимальное число нарушений направления. Ребро по направлению — стоимость 0, против — стоимость 1. 0-1 BFS: стоимость 0 -> appendleft, стоимость 1 -> append. K = максимум расстояний по всем стартовым вершинам.

Листинг 3 — task17.py

```

def find_min_k(n, edges):
    graph = [[] for _ in range(n+1)]
    for u, v in edges:
        graph[u].append((v, 0))
        graph[v].append((u, 1))
    max_k = 0
    for start in range(1, n+1):
        dist = [float("inf")]*(n+1)
        dist[start] = 0
        q = deque([start])
        while q:
            u = q.popleft()
            for (v, cost) in graph[u]:
                if dist[v] > dist[u]+cost:
                    dist[v] = dist[u]+cost
                    if cost==1: q.append(v)
                    else: q.appendleft(v)
            max_k = max(max_k, max(dist[1:n+1]))
    return max_k

```

●●● Рисунок 3 — Задача 17: Слабая K-связность

```

$ python3 task17.py
Тест 1: n=3, рёбра: 1->2, 1->3
От 2: до 1 и 3 нужно 1 нарушение -> K=1
output.txt: 1
Тест 2: n=4, цикл 2->4->1->3->2
Все вершины достижимы без нарушений -> K=0
output.txt: 0

```

Рисунок 3 — Демонстрация работы: Задача 17 (Слабая K-связность)

4.4*. Задача 13 — Грядки (дополнительное задание)

BFS/flood-fill по сетке $N \times M$. Для каждой непосещённой клетки "#" запускаем BFS, помечаем все связанные клетки "#" как посещённые (заменяем на "."), увеличиваем счётчик. Переходы только по горизонтали/вертикали. Сложность $O(N \times M)$.

Листинг 4 — task13_beds.py

```

def count_beds(N, M, grid):
    directions = [(-1,0), (1,0), (0,-1), (0,1)]
    count = 0
    for i in range(N):
        for j in range(M):
            if grid[i][j] == "#":
                count += 1
                queue = deque([(i,j)])
                grid[i][j] = "."
                while queue:
                    x, y = queue.popleft()
                    for dx, dy in directions:
                        nx, ny = x+dx, y+dy
                        if 0<=nx<N and 0<=ny<M and grid[nx][ny]=="#":
                            grid[nx][ny] = "."
                            queue.append((nx,ny))
    return count

```

●●● Рисунок 4 — Задача 13: Грядки

```

$ python3 task13_beds.py
Сетка 5x10:  ##.....#.
              .#..#....#.
              .###.....#.
              ..##.....#.
              .....#.

BFS 1: большая грядка слева
BFS 2: одиночная # (col=4, row=1)
BFS 3: вертикальная колонна (col=8)
output.txt: 3

```

Рисунок 4 — Демонстрация работы: Задача 13 (Грядки)

4.5*. Задача 15 — Герои (дополнительное задание)

BFS от позиции королевы по клеткам без засады (символ "0"). `time_grid[x][y]` — минимальное число ходов до (x,y). Для каждого мушкетёра (x,y,pendants): если `time_grid[x][y] <= L` — добавляем pendants к ответу.

Листинг 5 — task15_heroes.py

```
def calculate_pendants(N, M, grid, queen_pos, L, musketeers):
    directions = [(-1,0), (1,0), (0,-1), (0,1)]
    qx, qy = queen_pos
    time_grid = [[-1]*M for _ in range(N)]
    queue = deque([(qx, qy)])
    time_grid[qx][qy] = 0
    while queue:
        x, y = queue.popleft()
        for dx, dy in directions:
            nx, ny = x+dx, y+dy
            if 0<=nx<N and 0<=ny<M:
                if grid[nx][ny]=='0' and time_grid[nx][ny]==-1:
                    time_grid[nx][ny] = time_grid[x][y]+1
                    queue.append((nx,ny))
    total = 0
    for x, y, p in musketeers:
        if time_grid[x][y]!=-1 and time_grid[x][y]<=L:
            total += p
    return total
```

●●● Рисунок 5 — Задача 15: Герои

```
$ python3 task15_heroes.py
BFS от королевы (3,3):
time[1][1]=4, time[1][2]=3, time[2][1]=3, time[2][2]=2
L=10: все мушкетёры достигают -> 1+2+3+4=10
output.txt: 10
```

Рисунок 5 — Демонстрация работы: Задача 15 (Герои)

5. Вывод

В ходе лабораторной работы реализованы алгоритмы на графах: BFS для проверки достижимости в лабиринте (вывод 1/0), алгоритм Косарайю для подсчёта компонент сильной связности, 0-1 BFS для нахождения минимальной К-связности. Дополнительно реализованы flood-fill BFS для подсчёта грядок и многоисточниковый BFS для задачи мушкетёров. Все алгоритмы итеративные — устойчивы к переполнению стека.